

BOOTCAMP GENERATIVE AI · E2

Resumo do Dia 5

Sistemas Avançados

Material de consulta, compartilhado ao final do dia.

Dicas importantes do Dia 5

1. Só orquestre agentes quando um prompt único não resolve: cada passo é custo e risco a mais.
2. Coloque guard-rails em agentes: máximo de passos, ferramentas permitidas, confirmação humana para ações sensíveis.
3. Prefira frameworks prontos; só construa o seu quando houver razão forte (escala, segurança, controle).
4. Não escale antes de validar o MVP e estimar o custo em volume real.
5. Em produção, deixe observabilidade e segurança sempre ligadas: não são 'extras'.
6. Versione seus prompts como código: teste antes de subir mudanças.

Dia 5 • Sistemas Avançados

Sobre este resumo: Material complementar para quem vai além do MVP. Aqui você vê como orquestrar agentes em sistemas mais complexos, quando (e quando NÃO) desenhar um framework próprio, e como levar uma solução do protótipo à produção em escala. São temas de ‘escalar e produzir’: fazem sentido depois que você já tem um MVP funcionando.

Orquestração de agentes

Quando uma tarefa tem muitos passos ou precisa de várias especialidades, você orquestra agentes: um coordenador distribui sub-tarefas para agentes especializados (ou ferramentas) e junta o resultado. Frameworks como LangGraph, CrewAI e AutoGen ajudam a montar esses fluxos.

Por que importa: Orquestração resolve problemas que um único prompt não dá conta, mas adiciona complexidade, custo e pontos de falha. Use com critério.

Como aplicar / pontos-chave:

- Padrões comuns: pipeline (passos em sequência), roteador (escolhe o caminho), multi-agente (especialistas que colaboram).
- Cada passo do agente é uma chamada ao LLM: isso multiplica custo e latência.
- Defina limites: número máximo de passos, ferramentas permitidas, quando pedir confirmação humana.

Exemplo: *Atendimento: um agente classifica o pedido → roteia para o agente de cobrança, de troca ou de suporte técnico → cada um usa suas ferramentas → um agente final redige a resposta.*

Cuidado: Agente autônomo sem limites pode entrar em loop, gastar muito e tomar ações erradas. Sempre coloque guard-rails e um humano no loop em ações sensíveis.

Design de frameworks próprios

Em vez de usar um framework pronto (LangChain, etc.), times maduros às vezes constroem sua própria camada. Antes disso, entenda os componentes que qualquer solução de IA precisa: gestão de prompts, conexão com modelos, RAG/recuperação, memória/estado, ferramentas, avaliação e observabilidade.

Por que importa: Framework próprio dá controle e remove dependências, mas é caro de construir e manter. Na maioria dos casos, o pronto resolve.

Como aplicar / pontos-chave:

- Use pronto quando: você está começando, o time é pequeno, o caso é padrão.
- Considere próprio quando: requisitos muito específicos, escala alta, necessidade de controle/segurança que o pronto não dá.
- Mesmo com framework próprio, não reinvente tudo: reaproveite bibliotecas para as partes comuns.

Exemplo: *Uma empresa com requisitos rígidos de segurança e milhões de chamadas/dia pode justificar uma camada própria fina por cima das SDKs, não um framework do zero.*

Erro comum: Construir framework próprio cedo demais, ‘porque é legal’. Isso atrasa a entrega e cria manutenção eterna. Comece com o pronto.

Deploy em escala

Levar do protótipo à produção para muitos usuários: hospedar a aplicação (nuvem/containers), garantir disponibilidade, controlar custo em escala, e manter observabilidade e segurança ligadas.

Por que importa: O que funciona para 5 usuários quebra (ou fica caríssimo) para 5 mil. Escala exige cuidados de engenharia que o MVP não tinha.

Como aplicar / pontos-chave:

- Infra: containers (Docker) + nuvem (Azure/AWS/GCP) + orquestração (quando necessário).
- Custo em escala: cache, escolha de modelo por tarefa, limites de uso, monitorar gasto.
- Confiabilidade: timeouts, retries, fallback quando a API do modelo falha.
- CI/CD: testar mudanças (inclusive de prompt) antes de subir; versionar prompts.
- Observabilidade e segurança (Dia 3) ligadas o tempo todo.

Exemplo: Antes de abrir para a empresa toda, faça um piloto controlado, meça custo e latência reais, e só então escale com limites e alertas configurados.

Cuidado: Não escale antes de validar o MVP e estimar o custo em volume real: é assim que nascem as faturas-surpresa.

Leituras recomendadas • Dia 5

Para consolidar: pelo menos uma referência oficial por conteúdo do dia: papers e documentação. Clique para abrir.

- **Orquestração de agentes** • [AutoGen: Multi-Agent Conversation Framework · Wu et al. \(2023\) · arXiv:2308.08155](#)
- **Frameworks próprios** • [LlamaIndex · documentação oficial](#)
- **Deploy em escala** • [Production best practices · OpenAI \(guia oficial\)](#)

Vídeos recomendados · Dia 5

Curadoria para revisar o conteúdo do dia em vídeo: canais oficiais e referências (PT-BR e EN). Clique para abrir.

- **IA agêntica & orquestração** · [Generative vs Agentic AI · IBM Technology · EN](#)
- **Agentes em profundidade** · [What are AI Agents? · IBM Technology · EN](#)

Glossário do Dia 5

Orquestração • coordenar vários agentes/ferramentas em um fluxo

Multi-agente • vários agentes especializados que colaboram

Guard-rails • limites de segurança para o que a IA pode fazer

Framework • camada que organiza prompt, modelo, RAG, memória e ferramentas

Container • empacotamento da aplicação para rodar igual em qualquer ambiente

CI/CD • testar e publicar mudanças de forma automatizada